División del trabajo de programación (defensa 6)

Guía para repartir entre los **4 integrantes** la programación que pide la defensa 6, de forma que:

- todos tengan una parte propia, con commits reales y parecidos en cantidad;
- se pueda trabajar en paralelo sin pisarse los archivos;
- el código coincida 1 a 1 con el diagrama extendido, que la defensa 6 evalúa.

Se apoya en `Defensa 5/Correcciones finales/Guia-programacion-final.md` (la "guía de programación"), que tiene el código y las consideraciones de cada clase. Acá solo se dice **quién hace qué, en qué orden y cómo se integra**.

Los integrantes figuran como **Integrante 1 a 4**: reemplazar por los nombres al repartir.

Índice

- 1. Qué pide la defensa 6
- 2. Criterios del reparto
- 3. Antes de empezar: acuerdos de todo el equipo
- 4. El reparto en una tabla
- 5. Tareas de cada integrante
  - 5.1 Integrante 1: modelo de entidades
  - 5.2 Integrante 2: niveles, fábricas y recursos
  - 5.3 Integrante 3: vista y modos
  - 5.4 Integrante 4: motor, Link e hilos
- 6. Etapas y puntos de integración
- 7. Dependencias entre integrantes
- 8. Cómo trabajar con Git
- 9. Clean Code: reglas comunes
- 10. Revisión final antes de entregar
- 11. Qué explica cada uno en la defensa

---

1. Qué pide la defensa 6

Resumen del archivo `Requerimientos-defensa-6-2C-Zelda.pdf`.

Diagramas puntuables: reducido y extendido, con todos los patrones definidos, colisiones y uso de hilos. Ya están hechos: son los de `Defensa 5/Correcciones finales/`.

Programa del juego (implementación parcial):

Tipo	Requisito	Quién lo cubre en este reparto
Esencial a	Estructura de paquetes organizada con **todas** las clases e interfaces del juego completo	Los 4 (cada uno crea los esqueletos de sus clases)
Esencial b	Clean Code sostenido: indentación, nombres, métodos que hacen una sola cosa. **El código tiene que coincidir con los**	Los 4, con revisión cruzada

Tipo	Requisito	Quién lo cubre en este reparto
	diagramas, 1 a 1 en atributos y métodos	
Esencial c	Cargar al menos un nivel completo desde un archivo, y entregar `Formateo-de-niveles.pdf` junto al código (puede ir en el paquete del parser)	Integrante 2
Esencial d	Ver a Link, enemigos, plataformas y obstáculos sobre el nivel	Integrantes 2 y 3, con las entidades del 1
Esencial e	Que el estado de Link se vea en todo momento	Integrantes 3 y 4 (HUD con Observer)
Opcional f	Mover a Link en al menos una escena	Integrante 4
Opcional g	Control de colisiones parcial: ver cómo Link choca con las demás entidades	Integrante 4, con las entidades del 1

Condiciones de entrega:

- Los diagramas van en la misma carpeta que los de la defensa 4, **sin sobrescribirlos**, hasta las 8:00 del día de la defensa.
- El código va en el repositorio de la comisión, en una carpeta **al mismo nivel** que la de los diagramas (por ejemplo `Codigo/` junto a `Diagramas/`).

⚠️ **Importante:** "observar la estructura de paquetes que agrupe todas las clases" significa que tienen que existir **todas** las clases del extendido, aunque muchas tengan los métodos todavía vacíos. Y como se evalúa el 1 a 1, los esqueletos llevan **todos** los atributos y métodos del extendido, con los mismos nombres y tipos.

---

2. Criterios del reparto

1. **Por capas y paquetes.** Cada integrante es dueño de un grupo de paquetes. Así cada uno toca sus archivos y casi no hay conflictos de Git.
2. **Primero los contratos.** Las interfaces que unen las partes (`EntidadLogica`, `Observer`, `ControladorGrafico`, `ControlJugador`…) se escriben el primer día. Con eso cada uno programa contra la interfaz sin esperar al otro.
3. **Carga pareja.** El enunciado pide al menos un 20% del desarrollo por integrante. El reparto apunta a un 25% cada uno: algunos tienen más clases chicas y otros menos clases pero más difíciles.
4. **Cada uno con algo visible.** Todos tienen al menos un requisito esencial que se puede mostrar funcionando.
5. **Cada uno explica lo suyo.** En la defensa pueden preguntarle a cualquiera por cualquier parte, pero cada uno tiene que dominar la suya.

---

3. Antes de empezar: acuerdos de todo el equipo

Una reunión corta (1 hora) antes de escribir código. Al terminar, cada acuerdo queda escrito en el README del código.

#	Acuerdo	Propuesta
1	Estructura de paquetes	La de la sección 3 de la guía de programación (`vista`, `logica`, `logica.juego`, `logica.hilos`, `logica.mapa`, `logica.fabricas`, `logica.entidades`, `logica.entidades.enemigos`, `logica.entidades.fijas`, `logica.comportamientos`, `logica.util`, más la carpeta de recursos)
2	IDE y formateador	Todos el mismo (por ejemplo Eclipse o IntelliJ) con el **mismo** perfil de formato: tabulador o 4 espacios, llaves en la misma línea, líneas de 120 caracteres. Exportar el perfil y subirlo al repo
3	Versión de Java	La misma para todos (la que pida la cátedra)
4	Idioma de los nombres	Español, como en el diagrama. Sin abreviaturas
5	Carpeta del código	`Codigo/` al mismo nivel que `Diagramas/` en el repo de la comisión
6	Ramas y PR	Ver la sección 8
7	Decisiones abiertas	Revisar la sección 24 de la guía de programación (teclas, escudo, etc.) y dejarlas cerradas
8	Qué hacer si el diagrama no alcanza	Se avisa en el grupo, se decide entre todos y **se actualiza el extendido en el mismo cambio**. Nadie agrega atributos o métodos "por su cuenta"

Primer commit de cada integrante (el mismo día): el esqueleto del proyecto lo arma uno (por ejemplo el Integrante 3, que tiene `Principal`), y cada uno sube **sus** interfaces de contrato, para que todos tengan commits desde el principio:

Integrante	Interfaces que sube el primer día
1	`Colisionable`, `Colisionador`, `EntidadLogica`, `GeneradorDrop`, `GeneradorProyectil`
2	(no tiene interfaces) el enumerado `Direccion` y la clase `Posicion`, que todos usan
3	`Observer`, `ControladorGrafico`, `ControladorDeVistas`, y la clase `Principal` con el proyecto vacío
4	`ControladorJuego`, `ControlJugador`, `EntidadJugador`, `ComportamientoMovimiento`, `ComportamientoAtaque`, `ComportamientoDefensa`

---

4. El reparto en una tabla

Integrante	Área	Paquetes	Clases	Requisitos que muestra
1	Modelo de entidades	`logica.util` (salvo `Direccion` y `Posicion`), `logica.entidades` (base), `logica.entidades.fijas`, `logica.entidades.enemigos`	`Hitbox`, `Entidad*`, `Obstaculo*` y sus 5, `PowerUP*` y sus 4, `Objeto*`, `Espada`, `Escudo`, `Enemigo*`, `EnemigoConDrop*`, los 6 enemigos, `Jefe*`, `MasterStalfos`, `StoneHinox`, `Proyectil` y las estrategias de los enemigos	a, d, g (los efectos de cada choque)

Integrante	Área	Paquetes	Clases	Requisitos que muestra
2	Niveles, fábricas y recursos	`logica.mapa` (salvo `AdministradorColisiones`), `logica.fabricas`, `logica.juego` (ranking), recursos	`Direccion`, `Posicion`, `Sprite`, `Sector`, `Escena`, `Nivel`, `GeneradorNivel`, `FabricaEntidades`, `FabricaSprites*`, `FabricaSpritesOriginal`, `FabricaSpritesAlternativa`, `Ranking`, `EntradaRanking`, `PersistenciaRanking`; archivos de nivel, imágenes y `Formateo-de-niveles.pdf`	a, c, d
3	Vista y modos	`vista`, `logica` (contratos con la vista), `logica.juego` (modos)	`Principal`, `Ventana`, `GUI`, `PanelVista*`, `PanelPantallaPrincipal`, `PanelPantallaRanking`, `PanelPantallaNivel`, `CeldaGrafica`, `CeldaJugador`, `AnimadorTransicion`, `ControladorDeVistas`, `ControladorGrafico`, `Observer`, `ModoJuego*`, `Clasico`, `Contrarreloj`, `NoHit`	a, d, e
4	Motor, Link e hilos	`logica.juego` (`Juego`), `logica.hilos`, `logica.entidades` (animadas), `logica.comportamientos` (base)	`Juego`, `ControladorJuego`, `ControlJugador`, `EntidadJugador`, `HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`, `AccionProgramada`, `EntidadAnimada*`, `Link`, `AdministradorColisiones`, las 3 interfaces de Strategy, `MovimientoInducido`, `AtaquePorContacto`, `DefensaLimitada`	a, e, f, g

Estrategias de los enemigos (Integrante 1): `MovimientoAleatorio`, `MovimientoTeletransporte`, `AtaqueEspada`, `AtaqueRecargado`, `AtaquePorProyectil`, `AtaqueAvalancha`, `DefensaPorEscudo`. Para la defensa 6 alcanza con que existan completas en su firma; programarlas es un plus.

Por qué así:

- El **Integrante 4** tiene menos clases pero las más difíciles (hilos, `Juego`, Link, colisiones).
- El **Integrante 1** tiene muchas clases, pero casi todas son cortas y siguen el mismo molde.
- El **Integrante 2** tiene pocas clases y mucho trabajo de recursos: el archivo del nivel, las imágenes de los dos dominios y el PDF del formato.
- El **Integrante 3** tiene toda la parte Swing, que lleva tiempo de prueba y ajuste visual.

---

5. Tareas de cada integrante

Cada tarea dice qué entregar y cómo se comprueba. Las referencias "guía §N" son a secciones de la guía de programación.

5.1 Integrante 1: modelo de entidades

Objetivo: que existan todas las entidades del juego con sus atributos y métodos del extendido, y que los efectos de los choques estén programados.

#	Tarea	Guía	Terminado cuando
1.1	Interfaces `Colisionable`, `Colisionador`, `EntidadLogica`, `GeneradorDrop`, `GeneradorProyectil`	§8, §14.3, §14.4	Compilan (primer día)
1.2	`Hitbox` completo	§7.3	Un `main` de prueba: `intersecta`, `desplazada`, `adelante`, `estaDentroDe`, `estaEnLineaCon` dan lo esperado
1.3	`Entidad*` completa (con el cambio 4: `miSector` y `eliminar()`)	§8.4	Una entidad de prueba se agrega a un sector y se saca sola con `eliminar()`
1.4	`Obstaculo*` y sus 5 subclases	§9	`Roca` y `Agua` devuelven `bloqueaPaso()` en `true`
1.5	`PowerUP*` y sus 4, `Objeto*`, `Espada`, `Escudo`	§10, §11	Compilan con todos los métodos del extendido
1.6	Esqueletos de `Enemigo*`, `EnemigoConDrop*`, los 6 enemigos, `Jefe*`, `MasterStalfos`, `StoneHinox`, `Proyectil` (constructores con sus estrategias y valores)	§14	Se crean desde `FabricaEntidades` y se ven en el sector
1.7	Esqueletos de las 7 estrategias de enemigos	§15	Compilan con la firma del extendido
1.8	Efectos de choque: `afectar` de los obstáculos, `serRecogidoPor` de power-ups y objetos	§9, §10, §11	Con el Integrante 4: Link junta una rupia y suma puntos
1.9 (plus)	Movimiento aleatorio y ataque por contacto de un Moblin	§15.2	Un Moblin camina en el sector

Depende de: `Direccion` y `Posicion` (Integrante 2, día 1); `EntidadAnimada*` y `Link` (Integrante 4) para que compilen los enemigos y los efectos. Mientras tanto, puede escribir los esqueletos y comentar el código que llama a Link.

5.2 Integrante 2: niveles, fábricas y recursos

Objetivo: cargar el nivel 1 completo desde el archivo (requisito esencial c) con todas sus entidades.

#	Tarea	Guía	Terminado cuando
2.1	`Direccion` y `Posicion`	§7.1, §7.2	Compilan (primer día: los usan todos)
2.2	`Sprite`	§7.4	Prueba: `agregar`, `setEstado`, `getRutaActual`

#	Tarea	Guía	Terminado cuando
2.3	Imágenes de los dos dominios en `recursos/imagenes/original/` y `.../alternativa/`, con la convención de nombres `entidad-estado.png`	§17.2	Hay una imagen por cada estado que arma `FabricaSprites`
2.4	`FabricaSprites*`, `FabricaSpritesOriginal`, `FabricaSpritesAlternativa`	§17.2	Cada `crearSpritesX()` devuelve rutas que existen
2.5	`FabricaEntidades` (los 22 `crearX`)	§17.1	Cada método crea su entidad con sprite y juego
2.6	`Sector`, `Escena`, `Nivel`	§16.1 a §16.3	Pruebas de `agregar`, `quitar`, `getVecino`, `jefesDerrotados`
2.7	`GeneradorNivel` completo y `nivel1.txt` (y `nivel2.txt` si está listo)	§16.4	Un `main` de prueba carga el nivel 1 e imprime cuántas entidades hay por sector
2.8	`Formateo-de-niveles.pdf` en el paquete del parser	§16.4 (y `Defensa 3/Formateo-de-niveles.md`)	El PDF está junto a `GeneradorNivel` y coincide con el archivo real
2.9	`Ranking`, `EntradaRanking`, `PersistenciaRanking`	§18.5	Prueba: insertar 6 entradas deja las 5 mejores; guardar y volver a cargar da lo mismo

Depende de: las clases de entidades (Integrante 1) para que compile `FabricaEntidades`. Puede empezar por las fijas, que salen primero.

5.3 Integrante 3: vista y modos

Objetivo: que se vea el nivel con todas sus entidades y el HUD de Link actualizado (requisitos d y e), y el menú.

#	Tarea	Guía	Terminado cuando
3.1	Proyecto vacío, `Principal`, interfaces `Observer`, `ControladorGrafico`, `ControladorDeVistas`	§21.1, §18.2, §20.4	Compila y abre una ventana vacía (primer día)
3.2	`Ventana`, `PanelVista*`	§20.2, §20.3	Cambia de panel con `mostrarPanel`
3.3	`CeldaGrafica` y `CeldaJugador`	§20.5	Una entidad de prueba se dibuja en su posición y se borra al eliminarse
3.4	`PanelPantallaNivel` con el HUD	§20.3	Se ven las celdas en píxeles y las 6 etiquetas del HUD
3.5	`GUI` (todos los métodos de `ControladorGrafico` con `invokeLater`, y `procesarTecla`)	§20.1	Con el Integrante 4: al iniciar se ve el sector inicial del nivel 1
3.6	`PanelPantallaPrincipal` (elegir dominio y modo, jugar, ver ranking)	§20.3	Se puede iniciar una partida desde el menú en los dos dominios

#	Tarea	Guía	Terminado cuando
3.7	`ModoJuego*`, `Clasico`, `Contrarreloj`, `NoHit`	§18.4	Compilan; `getRanking(j)` devuelve el ranking de cada modo
3.8	`PanelPantallaRanking`	§20.3	Muestra el ranking de un modo
3.9 (plus)	`AnimadorTransicion` y `desplazar`	§20.6	El panel se desliza al cambiar de sector

Depende de: `EntidadLogica` y `EntidadJugador` (Integrantes 1 y 4, día 1), `ControladorJuego` (Integrante 4, día 1), `Ranking` (Integrante 2). Puede probar la vista con entidades de prueba antes de que exista `Juego`.

5.4 Integrante 4: motor, Link e hilos

Objetivo: que la partida arranque, cargue el nivel, muestre el sector y que Link se mueva y choque (requisitos e, f y g).

#	Tarea	Guía	Terminado cuando
4.1	Interfaces `ControladorJuego`, `ControlJugador`, `EntidadJugador` y las 3 de Strategy	§13.2, §13.3, §15.1, §18.1	Compilan (primer día)
4.2	`AccionProgramada`, `EntidadAnimada*`	§12	Compila; los enemigos del Integrante 1 heredan de ella
4.3	`MovimientoInducido`, `AtaquePorContacto`, `DefensaLimitada`	§15.2 a §15.4	Pruebas chicas de cada una
4.4	`HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`	§19	Un hilo de prueba imprime cada 30 ms y se pausa y detiene
4.5	`AdministradorColisiones`	§16.5	Prueba: una entidad no puede moverse sobre una roca
4.6	`Link` completo (con todos los métodos del extendido)	§13.1	Compila; los métodos de partida (puntos, vidas, inventario) funcionan
4.7	`Juego`: constructor, `iniciar`, `cargarNivel`, `activarSector`, `agregarEntidad`, `getControlJugador`, `tic`	§18.3	Con los Integrantes 2 y 3: la partida arranca, se ve el nivel 1 y el tiempo corre
4.8	Link camina con las flechas y lo frenan las rocas	§21.5	Requisito opcional f
4.9	Colisiones de Link con power-ups, obstáculos y enemigos	§16.5, §22	Requisito opcional g: Link junta una rupia y recibe daño de un Moblin
4.10 (plus)	Cambio de sector	§21.8	Link pasa del sector (0,0) al (0,1)

Depende de: las entidades (Integrante 1), el nivel (Integrante 2) y la vista (Integrante 3) para la integración. Hasta entonces, prueba con un sector armado a mano.

---

6. Etapas y puntos de integración

Las etapas son en orden; dentro de cada una, los 4 trabajan en paralelo. Cada etapa termina con un **punto de integración**: se juntan las ramas y se comprueba algo concreto.

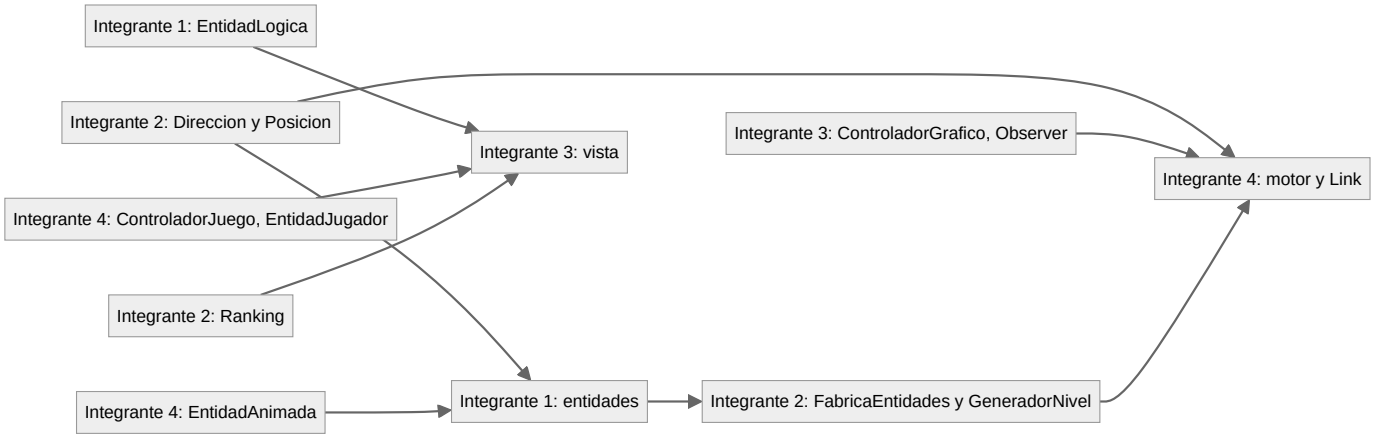
Etapa	Qué hace cada uno	Punto de integración (todos juntos)
0. Acuerdos	Reunión de la sección 3 y primer commit de cada uno (interfaces)	**P0:** el proyecto compila con las interfaces de los 4
1. Esqueletos	Cada uno crea **todas** sus clases con todos los atributos y métodos del extendido (cuerpos vacíos o `return` por defecto)	**P1:** compila el juego completo y se revisa entre todos el 1 a 1 con el extendido. **Esto ya cumple el requisito a**
2. Lo esencial	1: `Hitbox`, `Entidad`, fijas, constructores de enemigos. 2: sprites, fábricas, `GeneradorNivel`, `nivel1.txt`. 3: ventana, celdas, panel del nivel, HUD. 4: `EntidadAnimada`, `Link`, `Juego.iniciar/cargarNivel/activarSector`, hilos	**P2:** al ejecutar se ve el sector inicial del nivel 1 con Link, enemigos y obstáculos, y el HUD. **Requisitos c, d y e**
3. Lo opcional	1: efectos de choque. 2: `Formateo-de-niveles.pdf`, ranking. 3: menú, modos, ranking. 4: movimiento, `AdministradorColisiones`, colisiones de Link	**P3:** Link camina, lo frenan las rocas, junta una rupia (sube el puntaje en el HUD) y recibe daño de un enemigo. **Requisitos f y g**
4. Cierre	Revisión cruzada (cada uno revisa el código de otro), formateo, limpieza, README del código	**P4:** checklist de la sección 10 completa

Rotación de revisión cruzada (etapa 4): el 1 revisa al 2, el 2 al 3, el 3 al 4 y el 4 al 1. Cada uno revisa el 1 a 1 con el extendido y el Clean Code del otro, y deja comentarios en el PR.

---

7. Dependencias entre integrantes

Qué necesita cada uno de los otros y cuándo. Las flechas van de quien **provee** a quien **usa**.



Necesita	De quién	Qué	Para cuándo
Todos	Integrante 2	`Direccion`, `Posicion`	Etapa 0
Integrante 1	Integrante 4	`EntidadAnimada*` (al menos su esqueleto)	Etapa 1
Integrante 2	Integrante 1	Constructores de todas las entidades	Etapa 1
Integrante 3	Integrantes 1 y 4	`EntidadLogica`, `EntidadJugador`, `ControladorJuego`, `ControlJugador`	Etapa 0
Integrante 4	Integrante 3	`ControladorGrafico`, `Observer`	Etapa 0
Integrante 4	Integrante 2	`GeneradorNivel.generar` y `FabricaEntidades` funcionando	Etapa 2
Integrante 3	Integrante 2	`Ranking`	Etapa 3

Si alguien se atrasa: como todos programan contra interfaces, el que espera puede usar una clase de prueba propia (por ejemplo, una `EntidadLogica` falsa para probar las celdas) que **no se sube** al repositorio.

---

8. Cómo trabajar con Git

Regla	Detalle
Una rama por integrante y tarea	`prog/<nombre>-<tarea>`, por ejemplo `prog/ana-generador-nivel`
Commits chicos y con mensaje claro	Uno por clase o por funcionalidad: "Agregar GeneradorNivel con lectura del archivo". Nada de "cambios" o "arreglos"
Cada uno commitea desde su cuenta	El enunciado pide commits reales de cada integrante. Si uno programa en la computadora de otro, el commit igual lo hace el autor
Pull request a la rama principal	Otro integrante lo revisa antes de mezclarlo
Cada uno toca sus archivos	Si hace falta cambiar una clase de otro (por ejemplo, agregar un método), se avisa y lo hace el dueño, o se coordina en el PR
Integrar seguido	Traer los cambios de la rama principal al menos una vez por día (`git pull origin <rama-principal>`)
No subir	Archivos del IDE (`.idea/`, `.classpath`, `bin/`, `out/`), clases de prueba temporales, rankings guardados. Agregar un `.gitignore` en la etapa 0
Diagramas	Si un cambio de código obliga a tocar el extendido, el diagrama se actualiza en el **mismo PR**

Cantidad de commits esperada: con este reparto, cada integrante hace entre 15 y 30 commits hasta la defensa 6. Si alguno queda muy por debajo, se le pasan tareas de la lista "plus" de otro.

---

9. Clean Code: reglas comunes

La defensa 6 dice que el Clean Code "incidirá fuertemente en la nota" y que un código mal indentado se desaprueba. Estas reglas valen para los 4:

Regla	Ejemplo
Mismo formato en todo el proyecto	Pasar el formateador acordado antes de cada commit
Nombres que dicen qué hacen	`jefesDerrotados()`, no `check()`. Variables completas: `sector`, no `s`, salvo índices de un `for`
Un método, una cosa	Si un método necesita un comentario para separar partes, son dos métodos (y hay que agregar el segundo al extendido)
Métodos cortos	Idealmente menos de 20 líneas
Sin números mágicos sueltos	Los valores fijos de la guía (velocidades, tiempos) van en el constructor de cada clase con un nombre claro o como constante del extendido
Orden dentro de cada clase	Atributos, constructores, comandos, consultas (el mismo orden del extendido)
Sin código muerto	Nada comentado, ni `System.out.println` de prueba, ni `import` sin usar
Sin `instanceof`	Buscarlo antes de cada PR
Comentarios solo cuando aportan	Explicar el porqué, no repetir lo que dice el código

---

10. Revisión final antes de entregar

#	Control	Responsable
1	Compila desde cero en una computadora limpia (clonar el repo y ejecutar)	Integrante 4
2	Cada clase, atributo y método del extendido existe en el código, y no hay de más	Los 4, cada uno con sus paquetes; revisa el de al lado
3	Se carga el nivel 1 y se ve completo, con el HUD	Integrante 2
4	`Formateo-de-niveles.pdf` está en el paquete de `GeneradorNivel` y coincide con el archivo	Integrante 2
5	Los dos dominios gráficos se ven bien	Integrante 3
6	Link se mueve y choca (si llegaron los opcionales)	Integrante 4
7	Ningún `instanceof`, ni Swing fuera de la vista, ni `Timer`	Integrante 1
8	Formato aplicado a todos los archivos	Los 4
9	Diagramas de la defensa 6 subidos en la carpeta de los de la defensa 4, con nombre nuevo, antes de las 8:00	Integrante 3
10	Código en la carpeta al mismo nivel que la de diagramas	Integrante 4
11	Cada integrante tiene commits propios y repartidos	Los 4 (revisar el historial de GitHub)

---

11. Qué explica cada uno en la defensa

Integrante	Temas que tiene que dominar	Patrones y conceptos
1	Jerarquía de entidades, ganchos vacíos, cómo reacciona cada entidad a un choque, enemigos y sus estrategias	Visitor (doble despacho), Factory Method (`GeneradorDrop`, `GeneradorProyectil`), Strategy (configuración de cada enemigo)
2	Formato del archivo y parseo, sectores y escena, fábricas y dominios, ranking y persistencia	Abstract Factory (`FabricaSprites*`), Template Method (`armar` / `carpeta`), por qué el `switch` del parser no es `instanceof`
3	Separación lógica-vista, cómo un cambio en una entidad llega a la pantalla, HUD, EDT e `invokeLater`, menú y modos	Observer (`CeldaGrafica`), mapeo 1 a 1 entre entidad lógica y celda gráfica, polimorfismo de los modos
4	Hilos y por qué son dos más el reloj, una vuelta de cada hilo, `AdministradorColisiones`, la regla de los choques de Link, sincronización	Template Method (`HiloJuego`), Strategy (`setMovimiento`...), `AccionProgramada` sin hilos extra, abrazo mortal y orden de llaves

Pregunta que pueden hacerle a cualquiera: "¿qué pasa, clase por clase, cuando Link toca una rupia?". Está en la sección 10.2 de la guía de programación, y conviene que los 4 la sepan explicar.